LLM 部署优化

杜承翰 | NVIDIA 资深解决方案架构师 | 2023 年 01 月 18 日

Production Language Apps

Deploying massive models for real-time applications

- Increasing need for deep learning in language applications
  - Chat, translation, summarization, search, generation, etc.
- High accuracy models are important for correct results
  - Model accuracy directly correlates to helpfulness for users
- “Online” deployment require end-user acceptable latencies
  - Ensure a great experience with applications
- Multi-functional, accurate models are *large* making them slow during inference & expensive to deploy

Making **cost effective** deployments **challenging**

Large Language Model Ecosystem

Rapid evolution makes optimization challenging

- Increasing rate of new foundational LLMs being released
  - Llama, Falcon, Starcoder, ChatGLM, MPT, & more
- New operators & customization techniques makes optimization a moving target
- Latest models continue to be very large for best accuracy
  - 70-200 Billion parameter or more

Need a **performant**, **robust**, & **extensible solution** for **cost-effective**, **real-time** LLM deployments

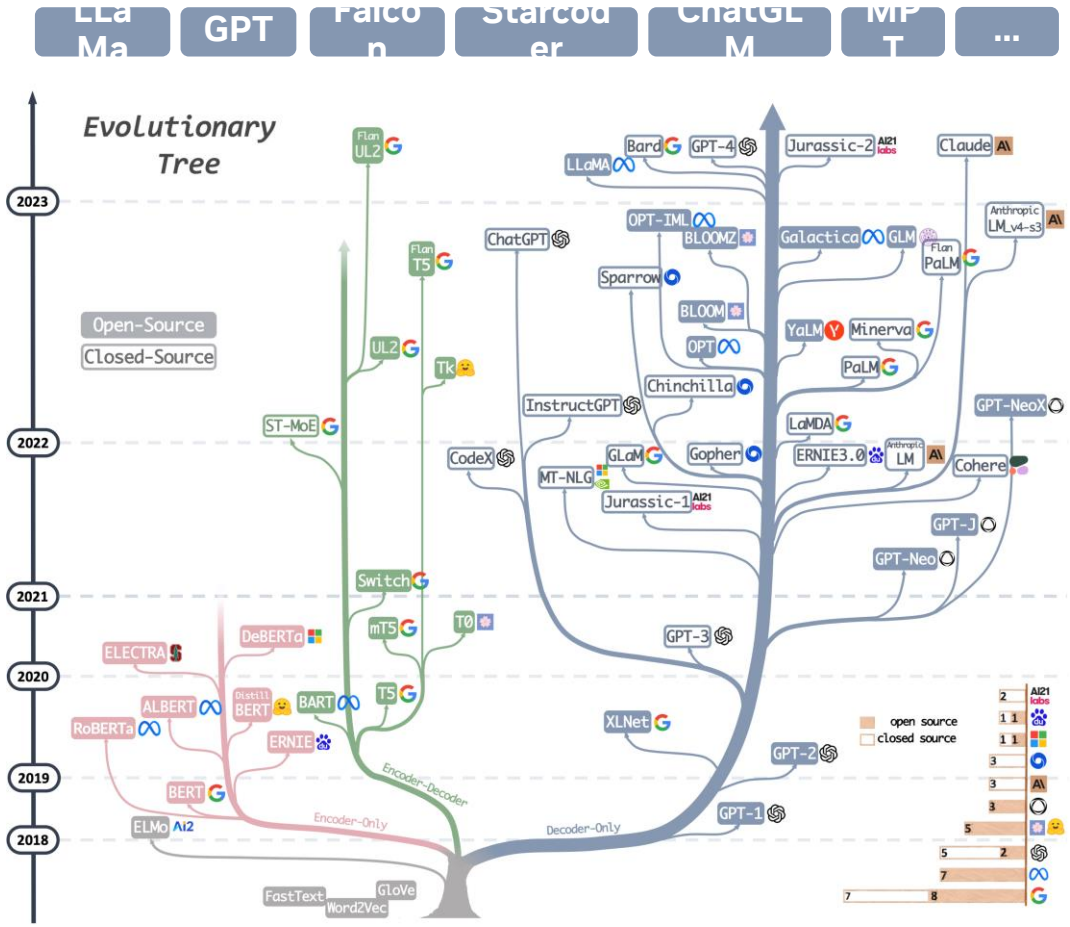


Image from Mooler0410/LLMsPracticalGuide

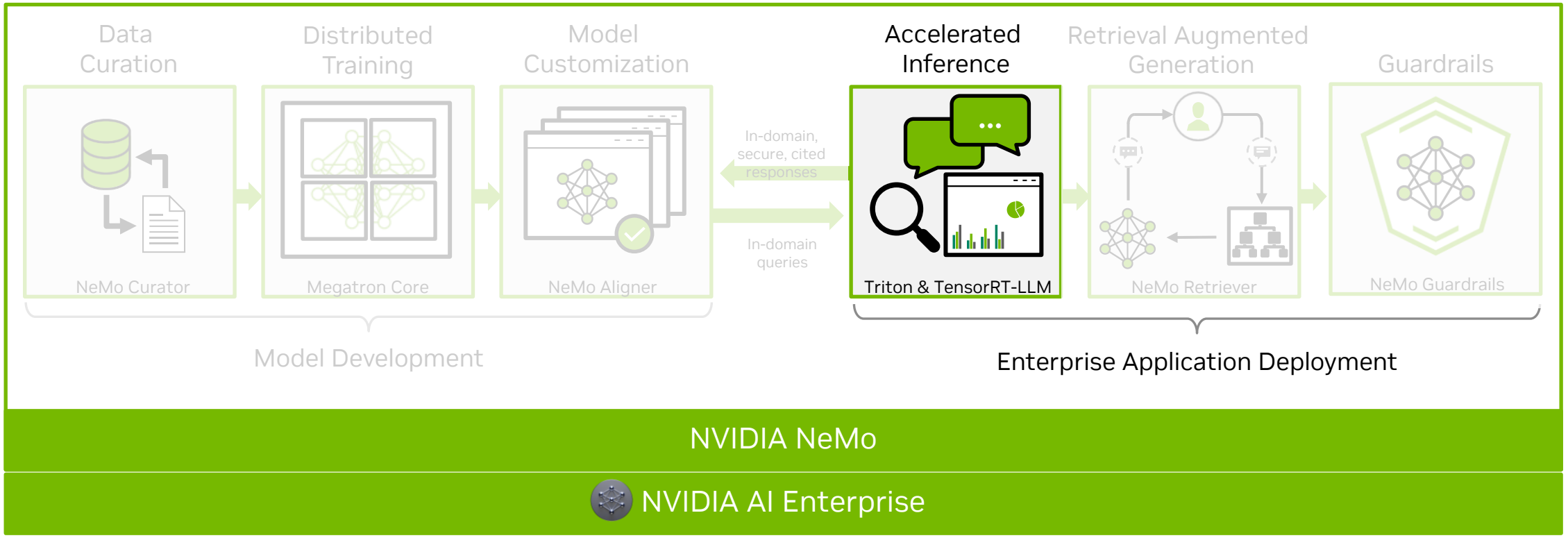
Yang, J., Jin, H., Tang, R., Han, X., Feng, Q., Jiang, H., ... Hu, X. (2023). Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond. arXiv [Cs.CL]. Retrieved from http://arxiv.org/abs/2304.13712

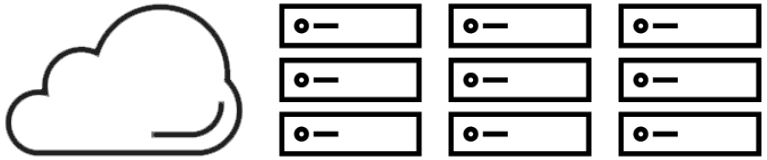
Agenda

- NVIDIA Containerized Solutions for LLM Inference
- Optimizing LLM Inference Service
- More Features for LLM Inference Service with Examples

Building Generative AI Applications for the Enterprise

Build, customize and deploy generative AI models with NVIDIA NeMo





Microsoft Azure · aws · ORACLE · NVIDIA DGX Cloud · DELL Technologies · Hewlett Packard Enterprise · Lenovo · SUPERMICRO

NVIDIA LLM Inference Stack in NeMo Framework Ecosystem

NeMo Framework: E2E model lifecycle

Guardrails, data curation, pretrained models, prebuilt e2e pipelines for deploying LLMs

Triton Inference Server: Model Serving

High Performance Inference Serving for AI model production deployments

TensorRT-LLM: Model Optimizer

Speed-of-Light LLM optimization

TensorRT

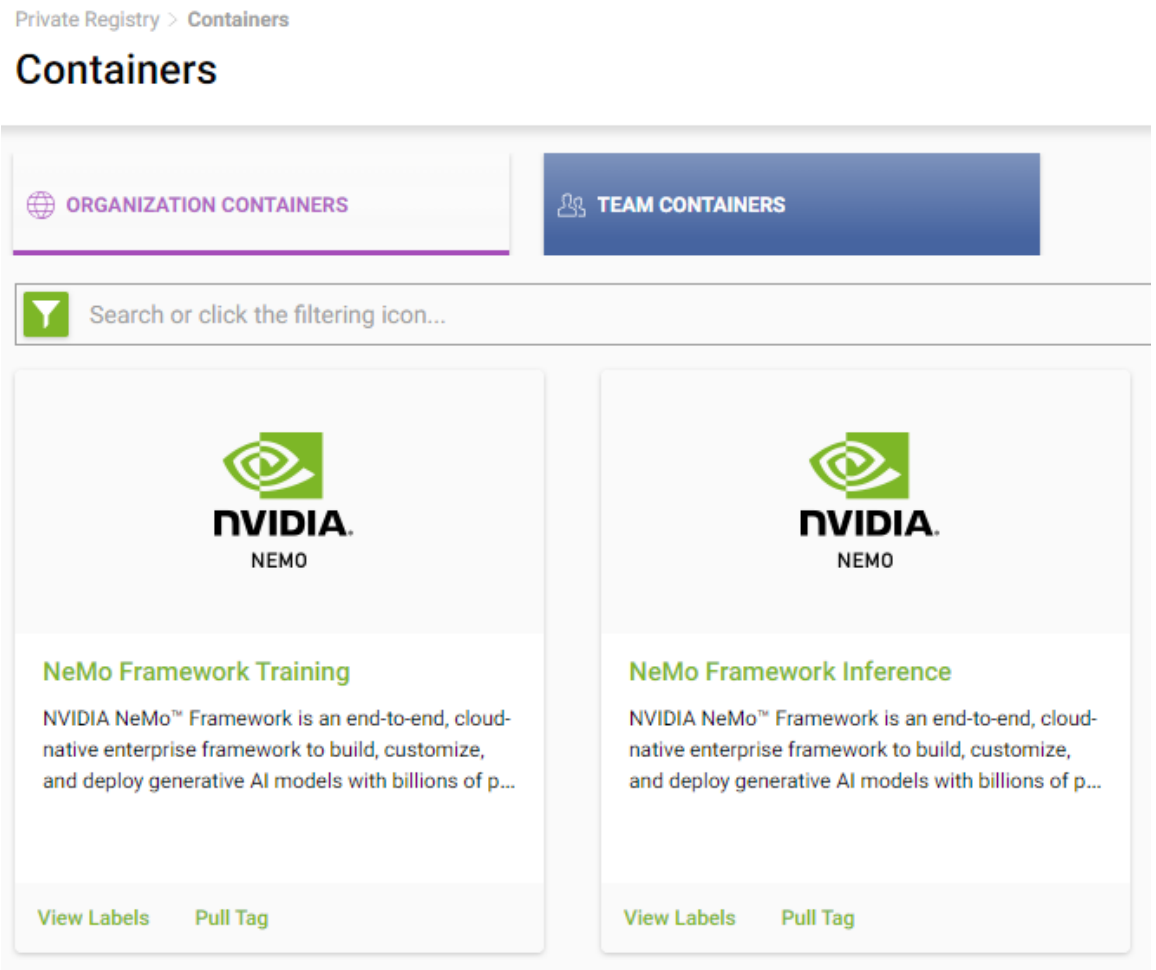
NVIDIA kernel libs

NVIDIA AI Enterprise

The Runtime for Production AI

NeMo Framework Inference

NeMo Framework Inference Container
- TensorRT-LLM development and runtime libraries included
- Additional model export scripts included



Available now! Apply for access
- https://developer.nvidia.com/nemo-framework
- https://docs.nvidia.com/nemo-framework/user-guide/latest/index.html

TensorRT-LLM Optimizing LLM Inference

SoTA Performance for Large Language Models for Production Deployments

Challenges: LLM performance is crucial for real-time, cost-effective, production deployments. Rapid evolution in the LLM ecosystem, with new models & techniques released regularly, requires a performant, flexible solution to optimize models.

TensorRT-LLM is an **open-source** library to **optimize inference performance** on the latest **Large Language Models** for NVIDIA GPUs. It is built on FasterTransformer and TensorRT with a simple Python API for defining, optimizing, & executing LLMs for inference in production.

SoTA Performance	Leverage TensorRT compilation & kernels from FasterTransformers, CUTLASS, OAI Triton, ++
Ease Extension	Add new operators or models in Python to quickly support new LLMs with optimized performance
LLM Batching with Triton	Maximize throughput and GPU utilization through new scheduling techniques for LLMs

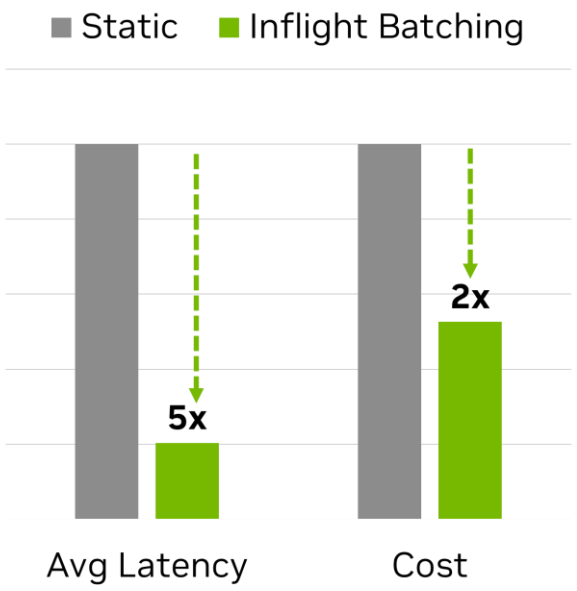
```
# define a new activation
def silu(input: Tensor) → Tensor:
    return input * sigmoid(input)

#implement models like in DL FWs
class LlamaModel(Module)
  def __init__(…)
    self.layers = ModuleList([…])

  def forward (…)
    hidden = self.embedding(…)

    for layer in self.layers:
      hidden_states = layer(hidden)

    return hidden
```

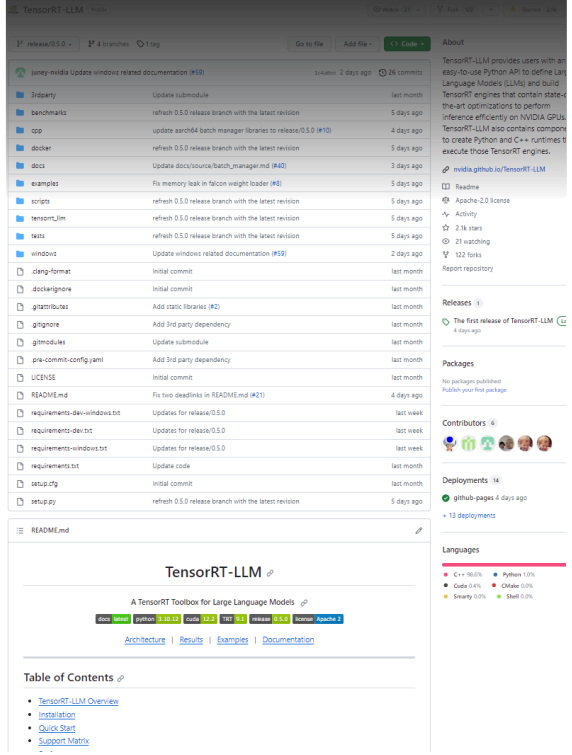


TensorRT-LLM Available Now!

Key resources for TensorRT-LLM

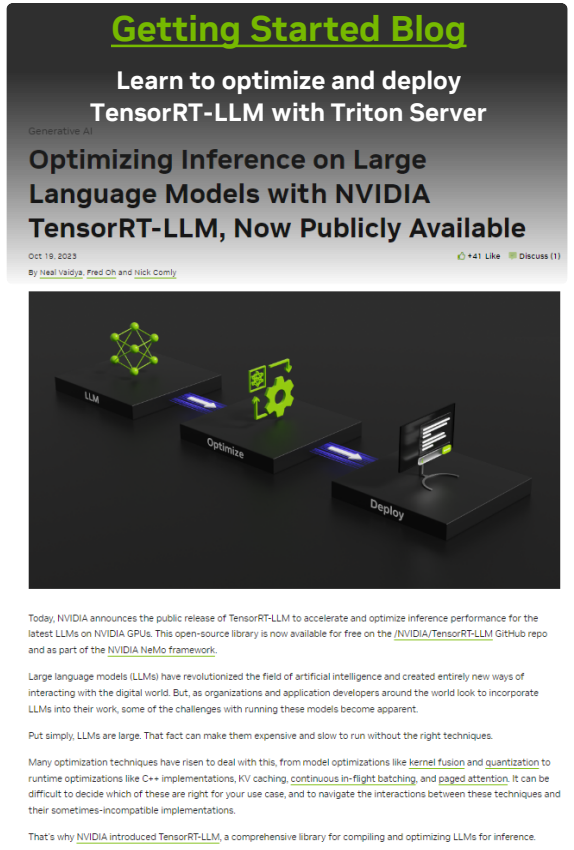
TensorRT-LLM Github

Source for TensorRT-LLM



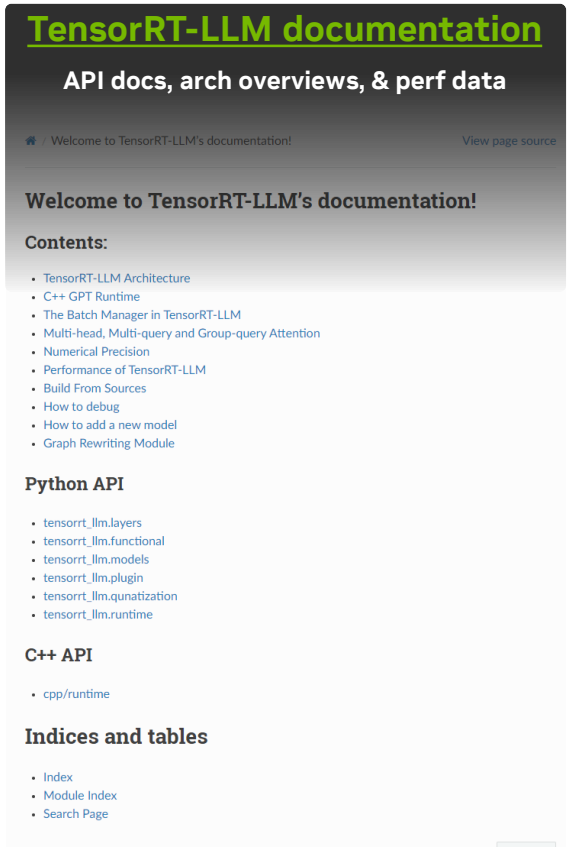
Getting Started Blog

Learn to optimize and deploy TensorRT-LLM with Triton Server



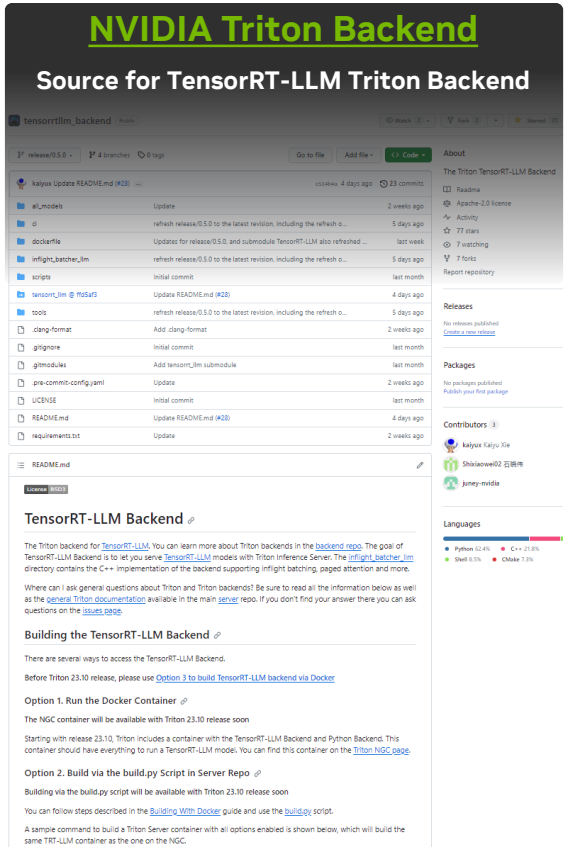
TensorRT-LLM documentation

API docs, arch overviews, & perf data



NVIDIA Triton Backend

Source for TensorRT-LLM Triton Backend



LLM Inference Solutions with Triton Inference Server

Option 1. Build via the `build.py` Script in Server Repo

Starting with Triton 23.10 release, you can follow steps described in the Building With Docker guide and use the build.py script.

A sample command to build a Triton Server container with all options enabled is shown below, which will build the same TRT-LLM container as the one on the NGC.

```
BASE_CONTAINER_IMAGE_NAME=nvcr.io/nvidia/tritonserver:23.10-py3-min
TENSORRTLLM_BACKEND_REPO_TAG=release/0.5.0
PYTHON_BACKEND_REPO_TAG=r23.10

# Run the build script. The flags for some features or endpoints can be removed if not needed.
./build.py -v --no-container-interactive --enable-logging --enable-stats --enable-tracing \
              --enable-metrics --enable-gpu-metrics --enable-cpu-metrics \
              --filesystem=gcs --filesystem=s3 --filesystem=azure_storage \
              --endpoint=http --endpoint=grpc --endpoint=sagemaker --endpoint=vertex-ai \
              --backend=ensemble --enable-gpu --endpoint=http --endpoint=grpc \
              --image=base,${BASE_CONTAINER_IMAGE_NAME} \
              --backend=tensorrtllm:${TENSORRTLLM_BACKEND_REPO_TAG} \
              --backend=python:${PYTHON_BACKEND_REPO_TAG}
```

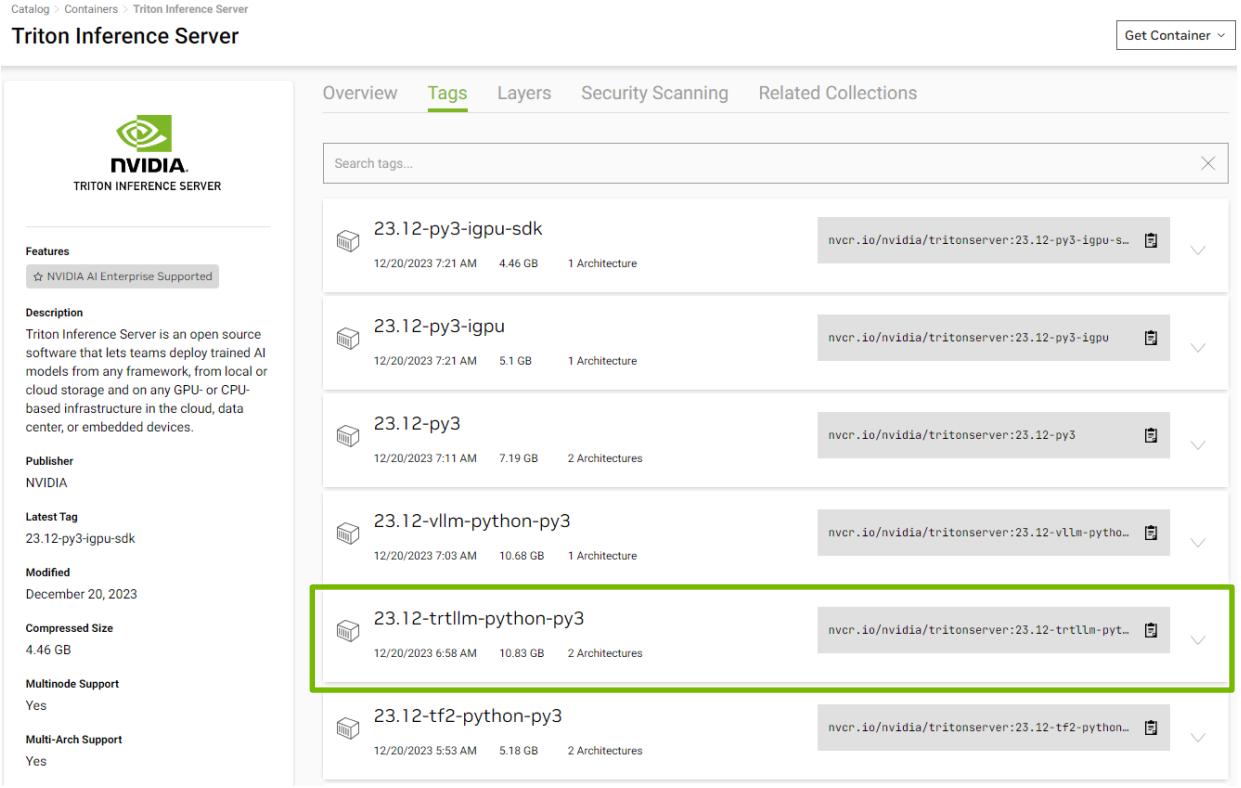
The `BASE_CONTAINER_IMAGE_NAME` is the base image that will be used to build the container. By default it is set to the most recent min image of Triton, on NGC, that matches the Triton release you are building for. You can change it to a different image if needed by setting the `--image` flag like the command below. The `TENSORRTLLM_BACKEND_REPO_TAG` and `PYTHON_BACKEND_REPO_TAG` are the tags of the TensorRT-LLM backend and Python backend repositories that will be used to build the container. You can also remove the features or endpoints that you don't need by removing the corresponding flags.

Option 2. Build via Docker

The version of Triton Server used in this build option can be found in the Dockerfile.

```
# Update the submodules
cd tensorrtllm_backend
git lfs install
git submodule update --init --recursive

# Use the Dockerfile to build the backend in a container
# For x86_64
DOCKER_BUILDKIT=1 docker build -t triton_trt_llm -f dockerfile/Dockerfile.trt_llm_backend .
# For aarch64
DOCKER_BUILDKIT=1 docker build -t triton_trt_llm --build-arg TORCH_INSTALL_TYPE="src_non_cxx11_abi" -f
```



Triton Inference Server with TensorRT-LLM in NGC

- Runtime library only
  - Useful when pre-built model engine already available
- Smaller image size

LLM Inference Solutions with Triton Inference Server

Option 1. Build via the `build.py` Script in Server Repo

Starting with Triton 23.10 release, you can follow steps described in the Building With Docker guide and use the build.py script.

A sample command to build a Triton Server container with all options enabled is shown below, which will build the same TRT-LLM container as the one on the NGC.

```
BASE_CONTAINER_IMAGE_NAME=nvcr.io/nvidia/tritonserver:23.10-py3-min
TENSORRTLLM_BACKEND_REPO_TAG=release/0.5.0
PYTHON_BACKEND_REPO_TAG=r23.10

# Run the build script. The flags for some features or endpoints can be removed if not needed.
./build.py -v --no-container-interactive --enable-logging --enable-stats --enable-tracing \
              --enable-metrics --enable-gpu-metrics --enable-cpu-metrics \
              --filesystem=gcs --filesystem=s3 --filesystem=azure_storage \
              --endpoint=http --endpoint=grpc --endpoint=sagemaker --endpoint=vertex-ai \
              --backend=ensemble --enable-gpu --endpoint=http --endpoint=grpc \
              --image=base,${BASE_CONTAINER_IMAGE_NAME} \
              --backend=tensorrtllm:${TENSORRTLLM_BACKEND_REPO_TAG} \
              --backend=python:${PYTHON_BACKEND_REPO_TAG}
```

The `BASE_CONTAINER_IMAGE_NAME` is the base image that will be used to build the container. By default it is set to the most recent min image of Triton, on NGC, that matches the Triton release you are building for. You can change it to a different image if needed by setting the `--image` flag like the command below. The `TENSORRTLLM_BACKEND_REPO_TAG` and `PYTHON_BACKEND_REPO_TAG` are the tags of the TensorRT-LLM backend and Python backend repositories that will be used to build the container. You can also remove the features or endpoints that you don't need by removing the corresponding flags.

Option 2. Build via Docker

The version of Triton Server used in this build option can be found in the Dockerfile.

```
# Update the submodules
cd tensorrtllm_backend
git lfs install
git submodule update --init --recursive

# Use the Dockerfile to build the backend in a container
# For x86_64
DOCKER_BUILDKIT=1 docker build -t triton_trt_llm -f dockerfile/Dockerfile.trt_llm_backend .
# For aarch64
DOCKER_BUILDKIT=1 docker build -t triton_trt_llm --build-arg TORCH_INSTALL_TYPE="src_non_cxx11_abi" -f
```

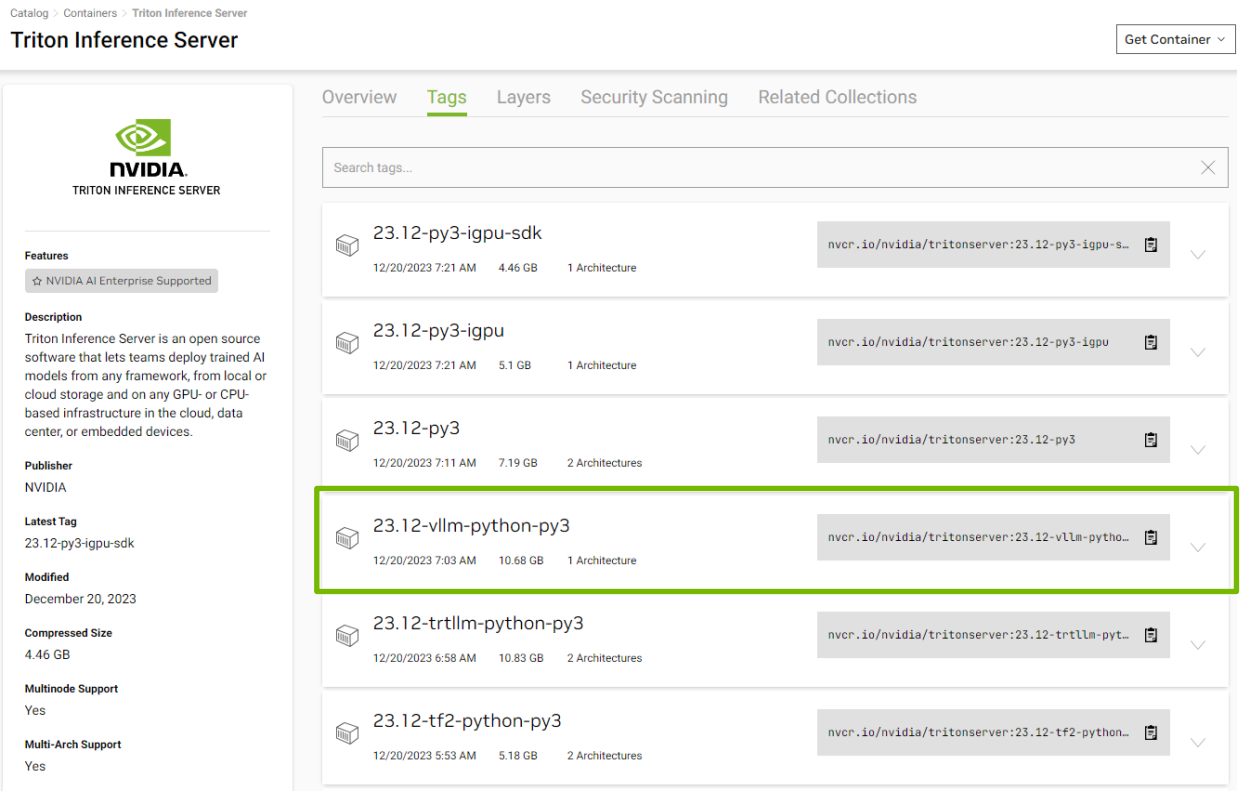
Alternative Build of Triton Inference Server with TensorRT-LLM

- Development and runtime libraries included
  - Build engine
- Useful for engine build and inference service

LLM Inference Solutions with Triton Inference Server

Third-party LLMs Solutions with Triton Inference Server

- Easier migration for vLLM user



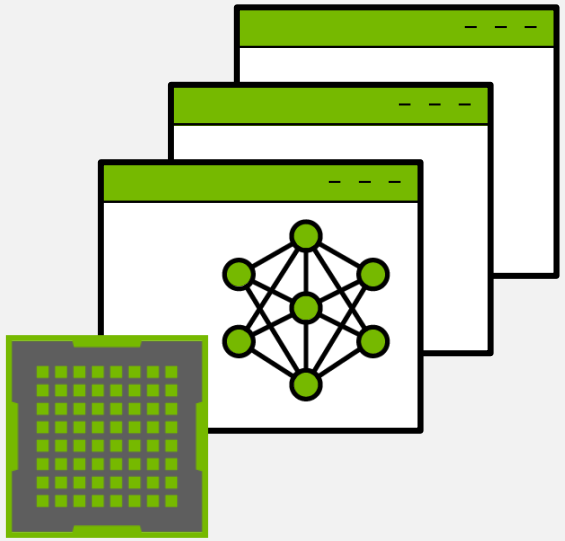
Agenda

- NVIDIA Containerized Solutions for LLM Inference
- Optimizing LLM Inference Service
- More Features for LLM Inference Service with Examples

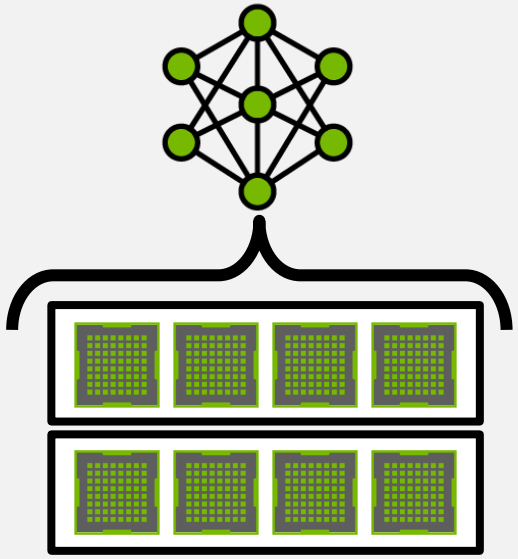
Deploying Large Scale Inference for Generative AI

Efficiently Deploy Generative AI Models At-scale With NeMo

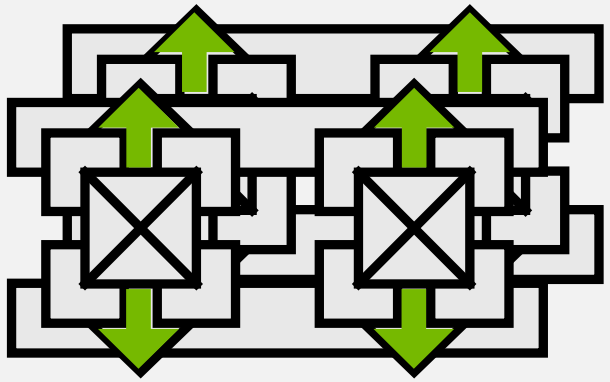
Optimized Kernels for Accelerated Performance



Multi-GPU and Multi-Node Inferencing



Intra/Inter-Node Communication

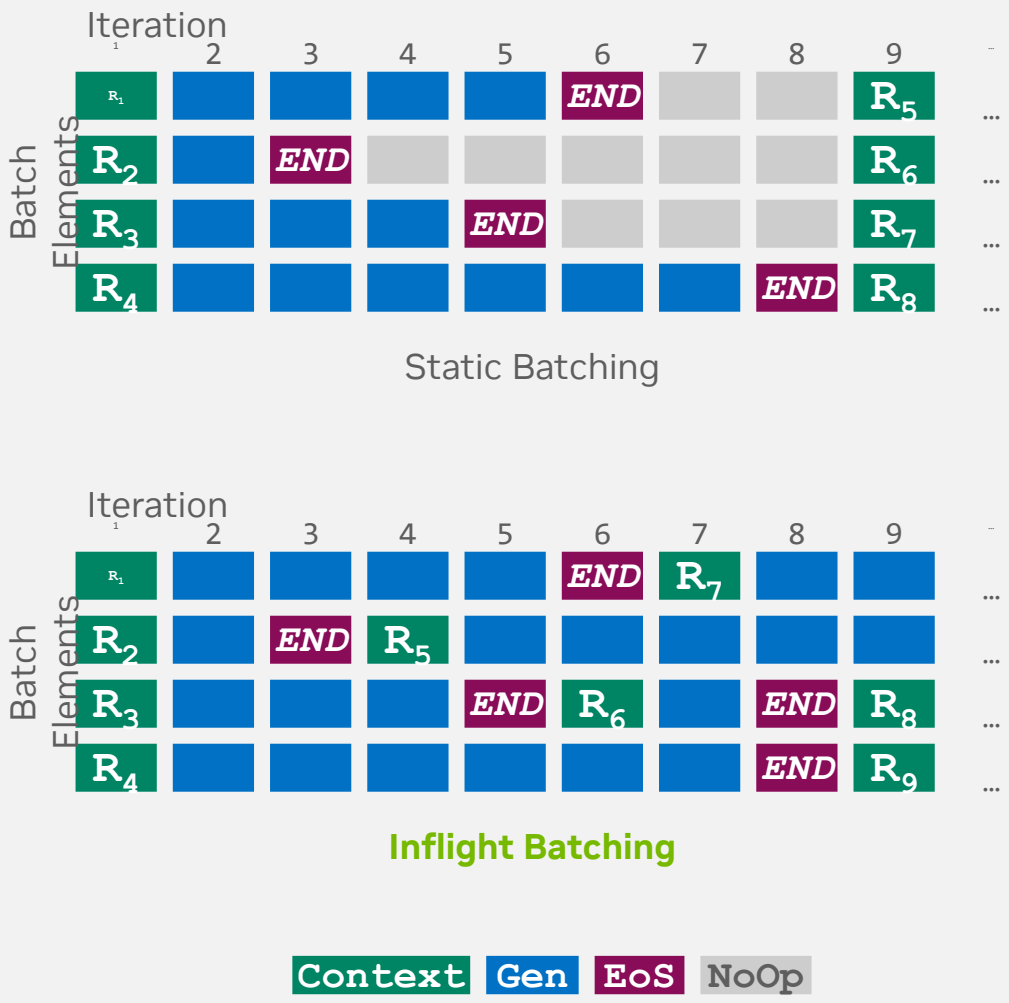


Inflight Batching

Maximizing GPU Utilization during LLM Serving

TensorRT-LLM provides custom Inflight Batching to optimize GPU utilization during LLM Serving

- Replaces completed requests in the batch
  - Evicts requests after EoS & inserts a new request
- Improves throughput, time to first token, & GPU utilization
- Integrated directly into the TensorRT-LLM Triton backend
- Accessible though the TensorRT-LLM Batch Manager



Optimization in TensorRT-LLM

Standard Dynamic Batching vs Inflight Batching

Serving with Inflight Batching: Comparison

Response from int4 quantized weights of Llama-2-13b-chat-hf with variable generation lengths. Maximum serving batch size is 4 for each server.

Question template: "Please provide one good name for a company that makes ___."

w/o Inflight batching

colorful socks

hair pins

furniture

mops

car window shields

liquid detergent

harmonica

winter gloves

Start multiple requests

Made with Streamlit

w/ Inflight batching

colorful socks

hair pins

furniture

mops

car window shields

liquid detergent

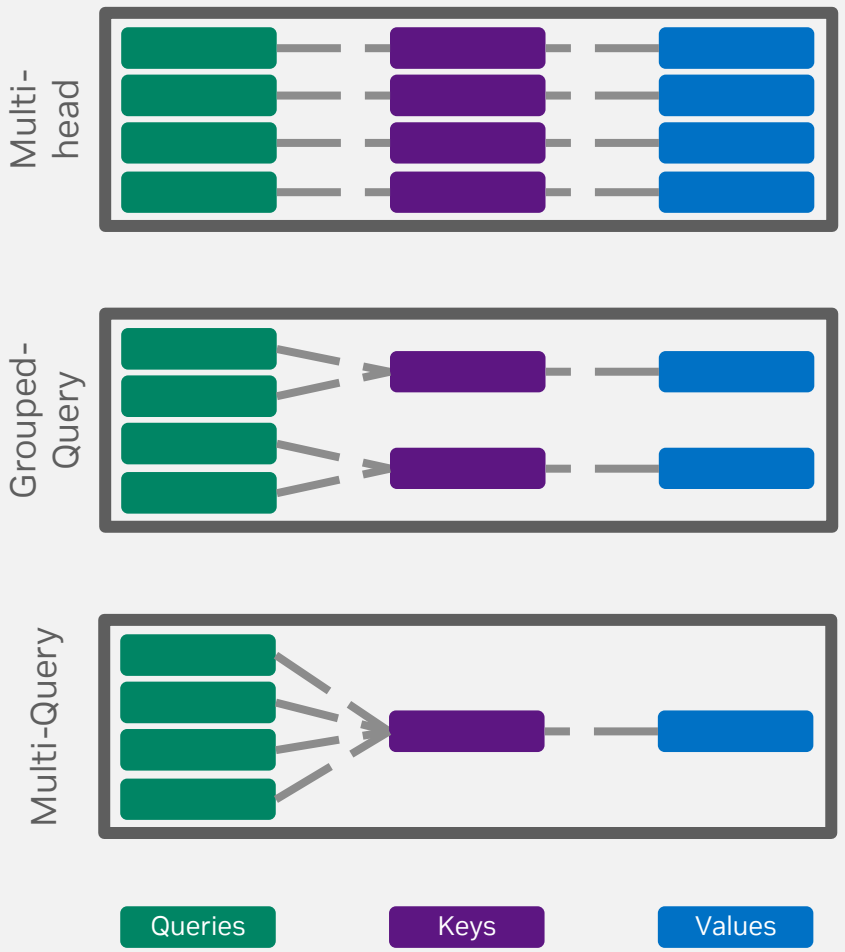
harmonica

winter gloves

Optimized Attention

Custom Implementations for Attention

- Custom optimized CUDA kernels for Attention
  - Similar to FlashAttentionV2
- Kernels for Encoder & Decoder, as well as context & prefill
- Supports MHA, MQA, GQA



KV Cache Optimizations

Paged & Quantized KV Cache

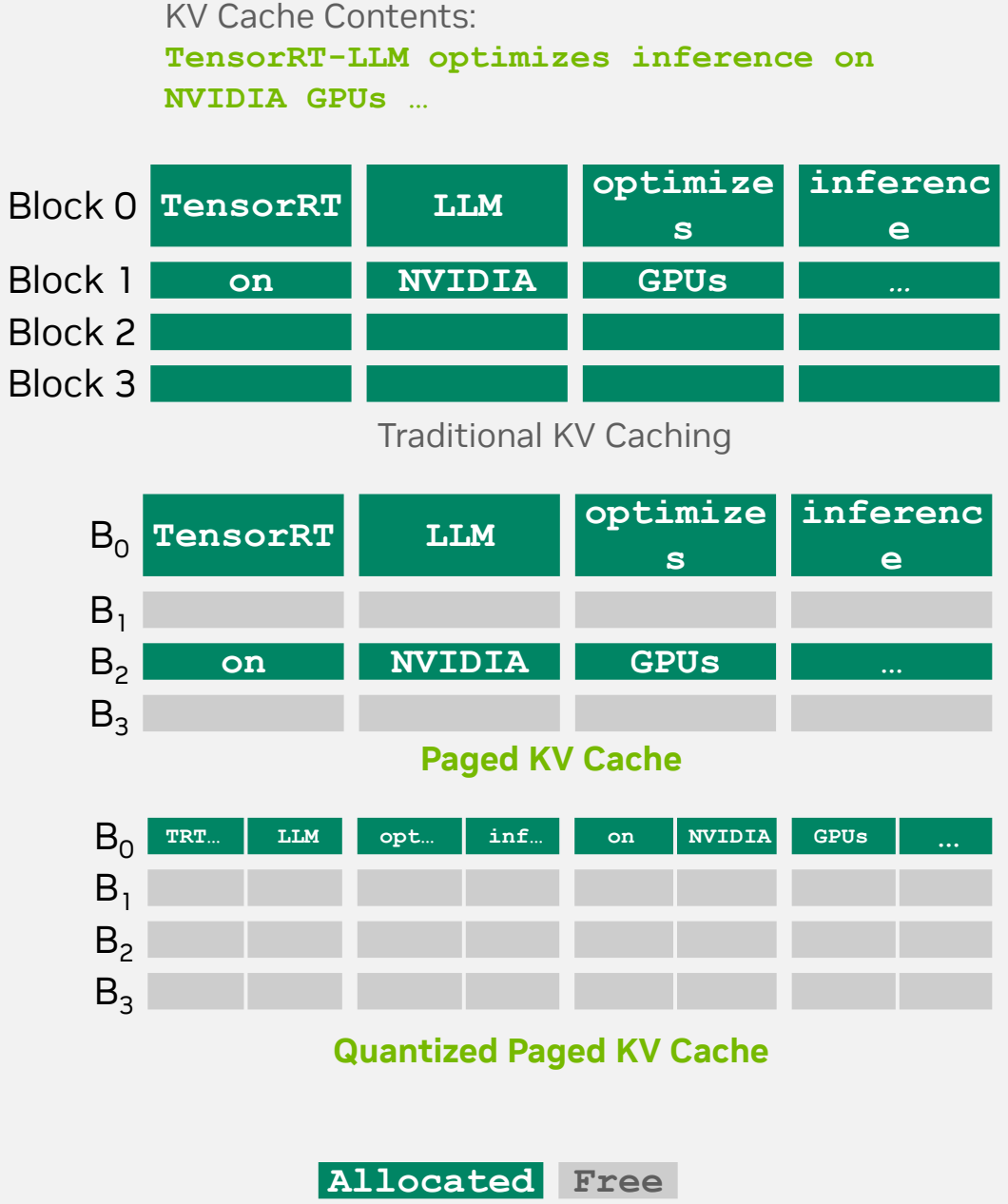
Paged KV Cache improves memory consumption & utilization

- Stores keys & values in non-contiguous memory space
- Allows for reduced memory consumption of KV cache
- Allocates memory on demand

Quantized KV Cache improves memory consumption & perf

- Reduces KV Cache elements from 16b to 8b (or less!)
- Reduces memory transfer improving performance
- Supports INT8 / FP8 KV Caches

Both allow for increased peak performance



Quantization

Supported Precisions & Models

- Utilizes Hopper FP8 “Transfomer Engine”
- Support many 8bit & 4bit methods
  - FP8, INT8/INT4 Weight only, INT8 Smooth Quant, AWQ, GPTQ
  - Support varies by model
- Reduced model size, memory bandwidth, & compute
  - Improves performance & allows for larger models per GPU
- Model optimization toolkit to quantize pre-trained models

	FP32	FP16	BF16	FP8	INT8	INT4
Volta (SM70)	Y	Y	N	N	Y	Y
Turing (SM75)	Y	Y	N	N	Y	Y
Ampere (SM80, SM86)	Y	Y	Y	N	Y	Y
Ada-Lovelace (SM89)	Y	Y	Y	Y	Y	Y
Hopper (SM90)	Y	Y	Y	Y	Y	Y

Model	FP32	FP16	BF16	FP8	W8A8 SQ	W8A16	W4A16	W4A16 AWQ	W4A16 GPTQ
Baichuan	Y	Y	Y	.	.	Y	Y	.	.
BERT	Y	Y	Y	.	.	.	.	.	.
BLOOM	Y	Y	Y	.	Y	Y	Y	.	.
ChatGLM	Y	Y	Y	.	.	.	.	.	.
ChatGLM-v2	Y	Y	Y	.	.	.	.	.	.
Falcon	Y	Y	Y	.	.	.	.	.	.
GPT	Y	Y	Y	Y	Y	Y	Y	.	.
GPT-J	Y	Y	Y	Y	Y	Y	Y	Y	.
GPT-NeMo	Y	Y	Y	.	.	.	.	.	.
GPT-NeoX	Y	Y	Y	.	.	.	.	.	Y
LLaMA	Y	Y	Y	.	Y	Y	Y	Y	Y
LLaMA-v2	Y	Y	Y	Y	Y	Y	Y	Y	Y
OPT	Y	Y	Y	.	.	.	.	.	.
SantaCoder	Y	Y	Y	.	.	.	.	.	.
StarCoder	Y	Y	Y	.	.	.	.	.	.

Optimization in TRT-LLM

Quantization for Better Throughput, Latency, Portability, and Scale

Llama-2 Quantization Comparison: FP16 v.s. INT4 Weights

FP16

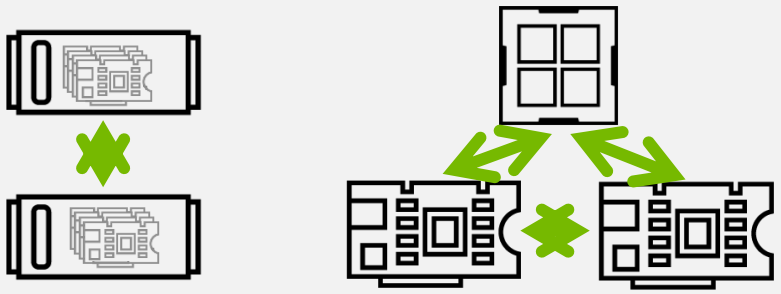
What is up?

INT4

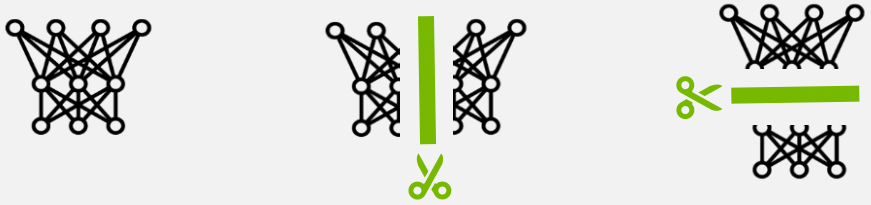
Multi-GPU Multi-Node

Sharding Models across GPUs

- Supports Tensor & Pipeline parallelism
- Allows for running very large models (tested up to 530B)
- Supports multi-GPU (single node) & multi-node
- TensorRT-LLM handles communication between GPUs
- Examples are parametrized for sharding across GPUs



Multi-Node Multi-GPU

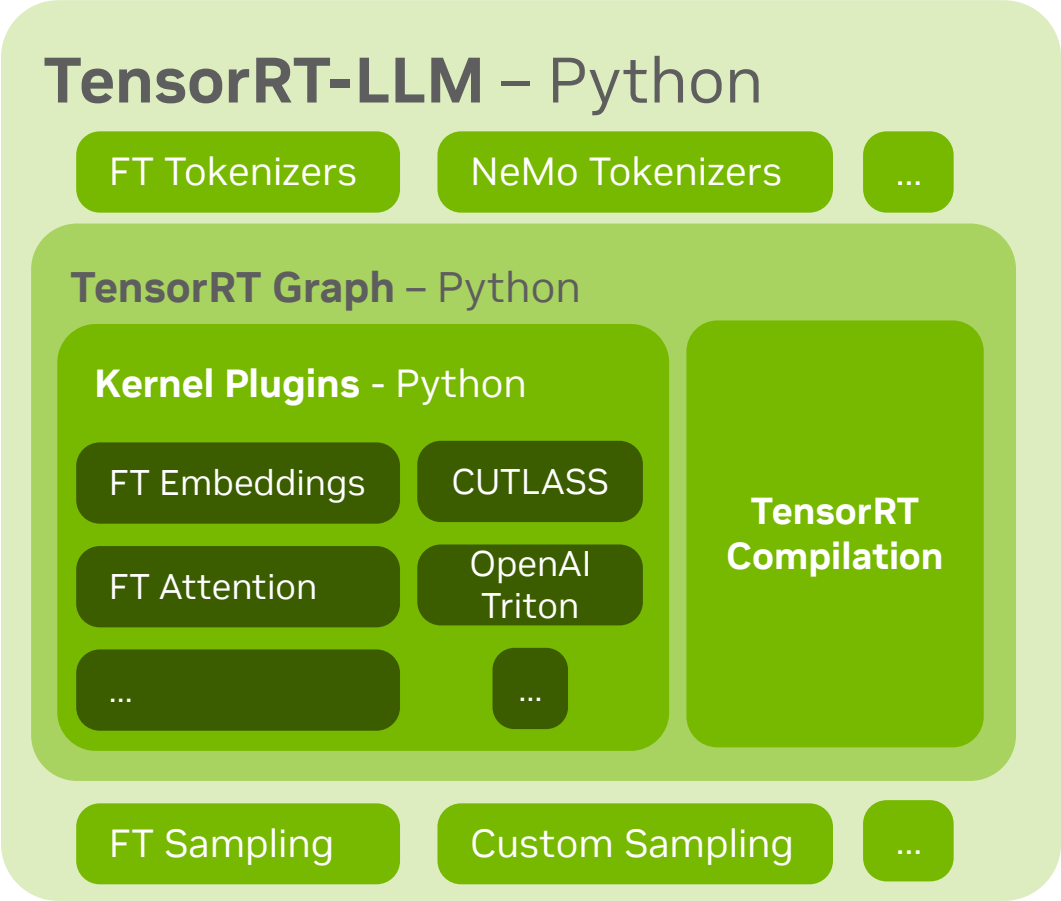


No Parallelism Tensor Parallel Pipeline Parallel

TensorRT-LLM

Leveraging the structure of TensorRT to modularize FT while preserving performance and versatility

- TensorRT-LLM leverages TensorRT as a framework for building and executing LLM graphs using FT kernels and optimizations
  - Wrapped in a framework-like API
    - *Iterate quickly with familiar FW-like APIs*
- TensorRT Engine is used for modularity and APIs
  - *Can use the same FT kernels as before*
- TensorRT compilation can be used for broad op coverage
  - *Map to TensorRT python APIs for primitives*
  - *LLMs performance is the main focus of TensorRT*
- Map layers to FT, TensorRT, OAIT, CUTLASS, or other sources
  - *Update to new operators entirely in-house*



TensorRT-LLM Execution

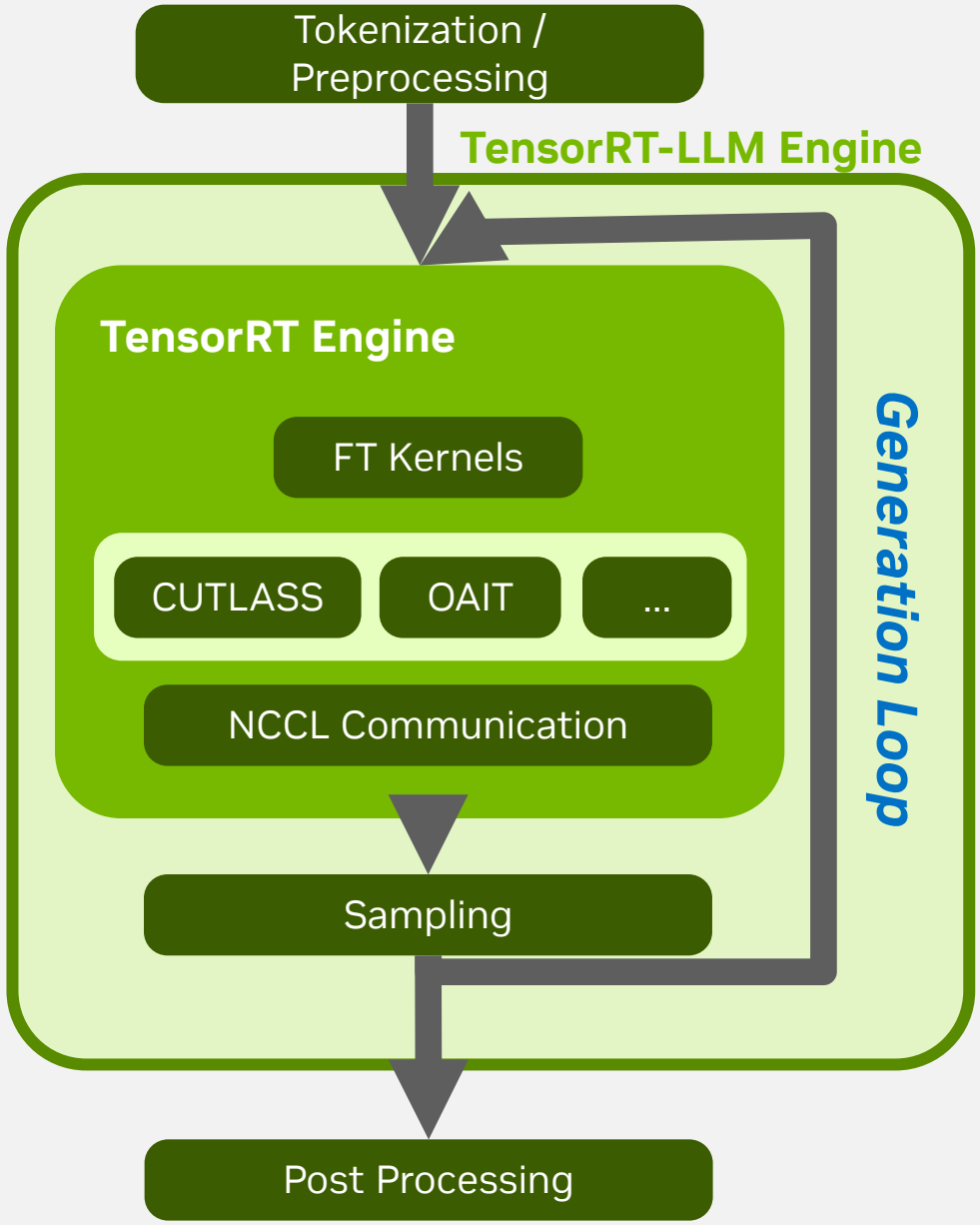
Built on top of FT & TensorRT

TensorRT Engine accelerates the Transformer Architecture

- Executes both TensorRT compiled graph and plugins
- Handles all plugin management & GPU communication

TensorRT-LLM handles the entire LLM

- Sampling runs on GPU for best performance loop
- Handles auto-regressive loop and batching
- Tokenization/detokenization can all be used directly

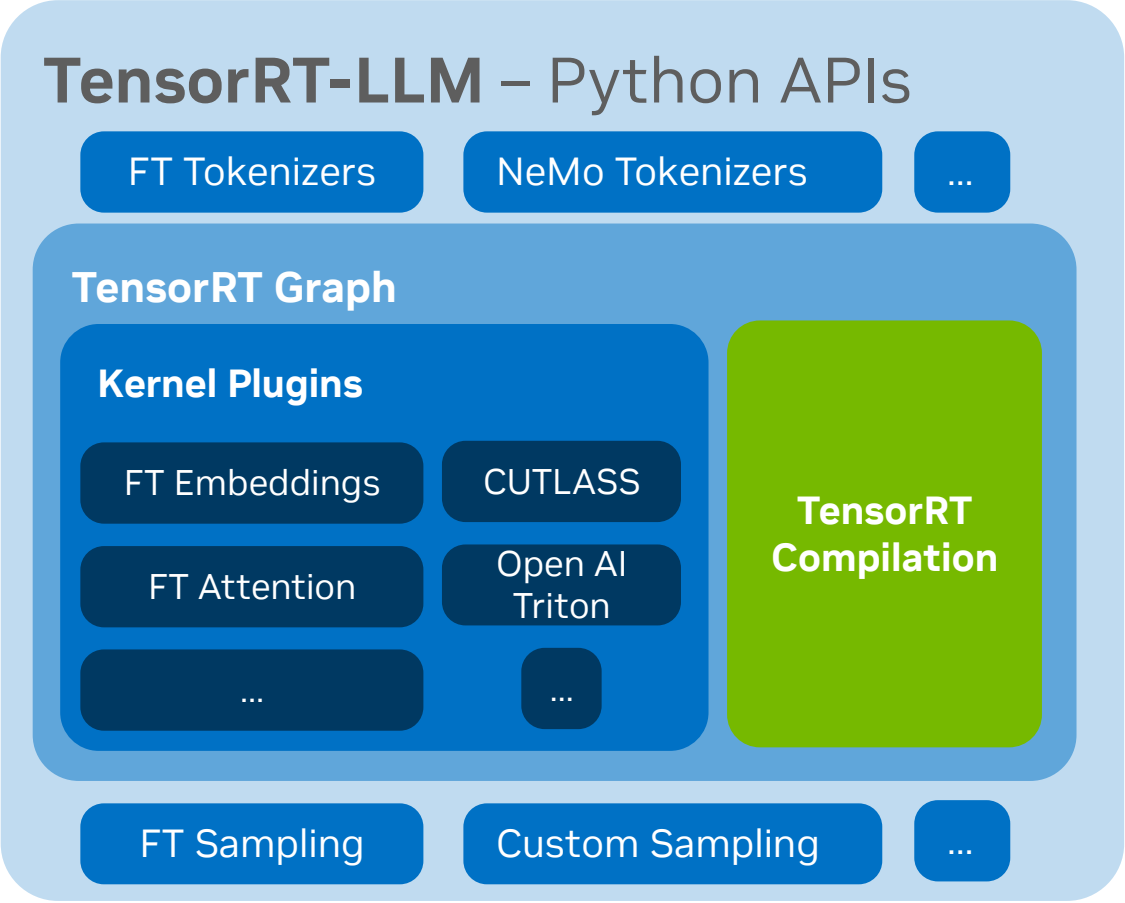


Open-Source Access

Maintaining access and “hackability” in TensorRT-LLM

- TensorRT-LLM will be open source under Apache2
  - Python APIs for model definition & weight loading
  - Existing kernels from FT
  - Pre/post-processing tokenizers & samplers will be OSS
  - Etc.
- Construct models with entirely OSS APIs and kernels

- TensorRT Compilation will remain closed source to the public



Open Source **Closed Source**

TensorRT-LLM Usage

Create, Build, Execute

- Instantiate model and load the weights
  - Load pre-built models or define via TensorRT-LLM Python APIs
- Build & serialize the engines
  - Compile to optimized implementations via TensorRT
  - Saved as a serialized engine
- Load the engines and run optimized inference!
  - Execute in Python, C++, or Triton

0. Trained Model in FW
NeMo, HuggingFace, or from DL Frameworks

1. Model Initialization
Load example model, or create one via python APIs

2. Engine Building
Optimized model via TensorRT and custom kernels

TensorRT-LLM Engine | **TRT Engine** | **Plugins**

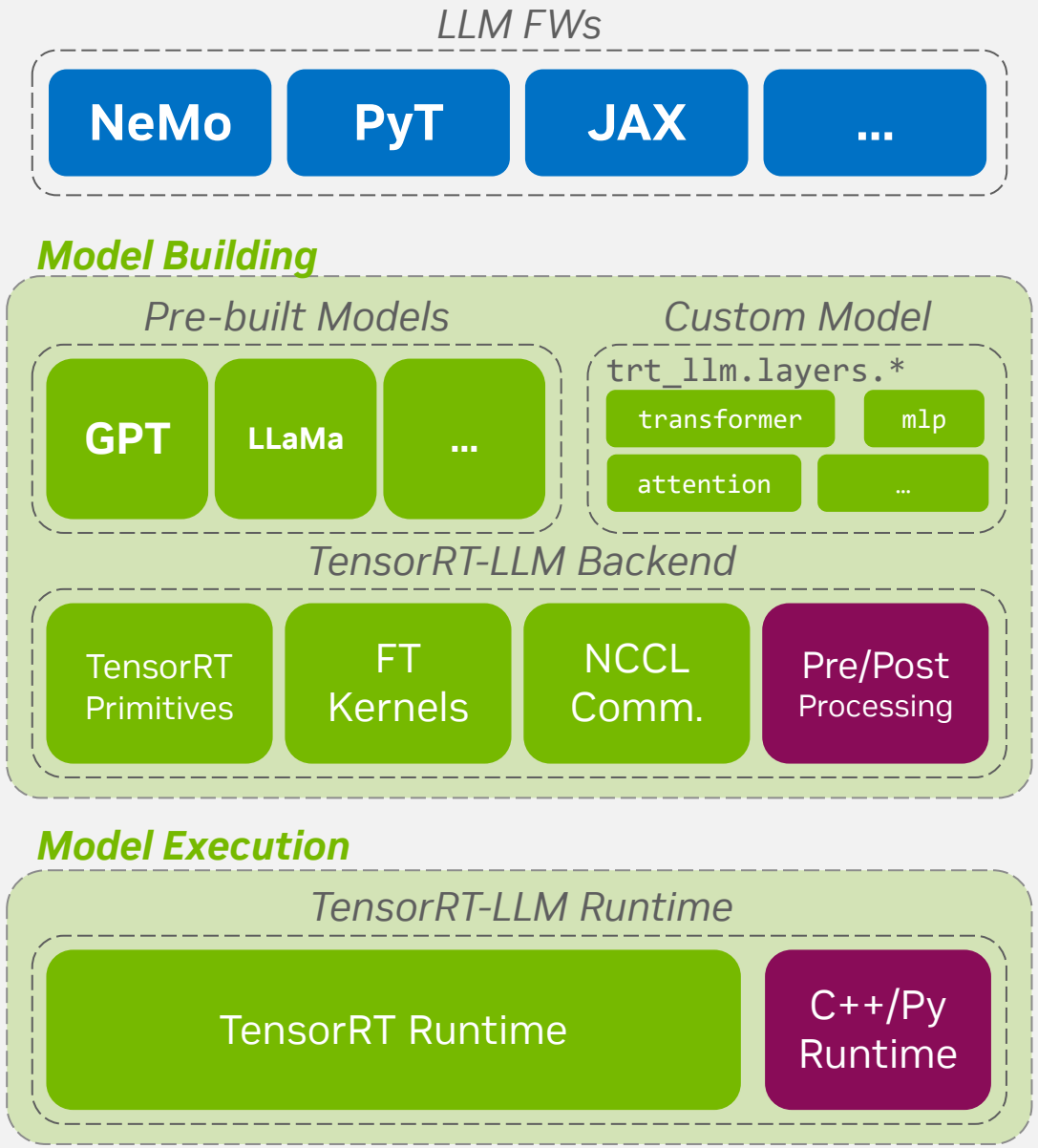
3. Execution
Load & execute in Triton Sever, C++, or Python

TensorRT-LLM Usage

Use Pre-built models, or optimize new ones!

TensorRT-LLM Inference

- Framework/TE-like building blocks for transformers
  - Ex. fMHA, layerNorm, activations, etc.
  - Built on top of TensorRT Python API
- Build arbitrary LLM or deploy pre-built implementations
  - Ex. GPT, LLaMa, BERT, etc.
- MGMN inference
  - Leverages NCCL plugins for multi-device communication
    - *Long term will be in TensorRT*
  - Pre-segmented graphs in pre-built models
  - User can manually segment for custom models
  - *Future will allow automatic segmentation across gpus*
- Combines TensorRT layers, NCCL plugins, perf plugins, & pre/post processing ops into a single object
  - Include tokenization & Sampling (ex. Beam search)



TensorRT-LLM Usage

Example: Running Llama in TensorRT-LLM

TensorRT-LLM Llama Getting Started Blog
TensorRT-LLM Examples

- Install and Build TensorRT-LLM from Github
- Download the pre-trained model
- Compile the model in TensorRT-LLM
- Run Optimized Inference!

```
$ git clone https://github.com/NVIDIA/TensorRT-LLM.git
$ cd TensorRT-LLM
$ git submodule update --init –recursive
$ git lfs install
$ git lfs pull
$ make -C docker release_build
```

Install & Build TensorRT-LLM

```
$ git lfs install
$ git clone https://huggingface.co/meta-llama/Llama-2-7b-chat-hf
```

Download the Model

```
$ make -C docker release_run LOCAL_USER=1

$ huggingface-cli login --token *****

$ python3 examples/llama/build.py \
    --model_dir meta-llama/Llama-2-7b-chat-hf \
    --dtype float16 \
    --use_gpt_attention_plugin float16 \
    --use_gemm_plugin float16 \
    --output_dir examples/llama/out
```

Compile the Model

```
$ python3 examples/llama/run.py --engine_dir=examples/llama/L40 --
max_output_len 100 --tokenizer_dir meta-llama/Llama-2-7b-chat-hf --
input_text "How do I count to nine in French?"
```

Run the Model

Implementing New Operators

Utilizing custom CUDA kernels

TensorRT-LLM can use custom kernels via “Plugins”

- Allows for peak performance on key ops (ex. MHA)
- Quickly improve any performance bottlenecks
- Any kernels from CUTLASS, OpenAI Triton, CUDA, & more
- Insert as layers directly into the TensorRT-LLM model
- Execution and management handled entirely by TensorRT

0. GPU Kernel
Compiled GPU kernel from CUTLASS, OAIT, or other

1. Define kernel config
Metadata on the kernel for plugin generation

2. Generate the plugin!
TensorRT-LLM auto generates the plugins

TensorRT-LLM Plugin | `functional.py` layers

3. Embed!
Use the generated layers in the model definition

TensorRT-LLM Usage

Model Building & Defining Operators

Easily modify models

- Modify the model similar to in a DL FW
- Add operators to forward call as desired
- Ops can be used with any model

Improved op coverage & definition

- Define ops in Python with TensorRT Python primitives
- *or* Map ops to arbitrary kernels with plugins
- Compose operators in Python

```
# llama.py
class LLaMaModel(Module)
  def __init__(…)
    self.layers = ModuleList([…,trt_llm.linear()])

  def forward (…)
    hidden_states = self.embedding(…)

    for layer in self.layers:
      hidden_states = layer(hidden_states)

    # add a new layer to a model with modular building blocks.
    hidden_states = self.linear(hidden_states)
    return hidden_states
```

Modify models simply with modular Python-layers

```
# functional.py
def sigmoid(input: Tensor) → Tensor:
  layer = default_trtnet().add_activation(input.trt_tensor,
                                           trt.ActivationType.SIGMOID)
  return _create_tensor(layer.get_output(0), layer)


def silu(input: Tensor) → Tensor:
    return input * sigmoid(input)


def swiglu(input: Tensor) -> Tensor:
    x, gate = chunk(input, 2, dim=-1)
    return silu(gate) * x
```

Creating new ops from TRT is only a few lines of Python

Implementing New Operators

RMSNorm via TensorRT Python APIs

Implement new operators quickly via TensorRT

- Quickly implement, entirely in python, unblock deployments
- Compiled entirely in TensorRT
- Fused into a single kernel
- Same or similar performance to custom CUDA kernels

TensorRT may not always fuse operators to a single kernel, impacting performance.

```
def rms_norm(input: Tensor,
             normalized_shape: Union[int, Tuple[int]],
             weight: Optional[Tensor] = None,
             eps: float = 1e-05) → Tensor:
  dim = tuple([-i - 1 for i in range(len(noramlized_shape))])

  with precision(“float32”)
    varx = pow(input, 2.0)
    varx = varx.mean(dim, keepdim=True)
    denom = varx + eps
    denom = denom.sqrt()
    y = input / denom

  if weight is not None:
    y = y * weight

  return y
```

Implementing RMSNorm in TensorRT-LLM

Agenda

- NVIDIA Containerized Solutions for LLM Inference
- Optimizing LLM Inference Service
- More Features for LLM Inference Service with Examples

TensorRT-LLM Release Process

Releases Monthly, Dev Updates Weekly

`main/`

- Updated ~ weekly
- Includes features as they are completed
- Includes preview & experimental features
- CI/CD, testing, & code reviews before publishing
- Recommended for the experimentation & development
- Release notes available in **Discussions**

`rel/`

- Updated ~ monthly
  - Versions are tagged on Github
- Branched from `main/` from ~ 2 weeks prior
  - Will contain additional bug fixes & tweaks
- Fully QA'd, security tested, & benchmarked
- Recommended for final integrations & production usage
- Release Notes available in **Releases**

Rapid Expansion of New Features

Contribution from Developer Community

Initial Release (v0.5.0)

Baichuan, BLIP2, BERT, BLOOM, ChatGLM, Falcon, GPT/J/NeoX, Llama v1/v2, MPT, OPT, Santa/StarCoder

Contains features from main/ ~2 weeks prior

Oct. 19 2023

v0.6.1

- Models
  - ChatGLM3
  - InternLM (contributed by @wangruohui)
  - Mistral 7B (developed in collaboration with Mistral.AI)
  - MQA/GQA support to MPT (and GPT) models (contributed by @bheilbrun)
  - Qwen (contributed by @Tlntin and @zhaohb)
  - Replit Code V-1.5 3B (contributed by @bheilbrun)
  - T5, mT5, Flan-T5 (Python runtime only, contributed by @mlmonk and @nqbao11)
- Features
  - Add runtime statistics related to active requests and KV cache utilization from the batch manager
  - Add `sequence_length` tensor to support proper lengths in beam-search (when beam-wi 1 -
  - BF16 support for encoder-decoder models (Python runtime - see examples/enc_dec)
  - Improvements to memory utilization (CPU and GPU - including memory leaks)
  - Improved error reporting and memory consumption
  - Improved support for stop and bad words
  - INT8 SmoothQuant and INT8 KV Cache support for the Baichuan models (see examples/baichuan

Dec. 4 2023

v0.7.1

- Models
  - BART and mBART support in encoder-decoder models
  - FairSeq Neural Machine Translation (NMT) family
  - Mixtral-8x7B model
    - Support weight loading for HuggingFace Mixtral model
  - OpenAI Whisper
  - Mixture of Experts support
  - MPT - Int4 AWQ / SmoothQuant support
  - Baichuan FP8 quantization support
- Features
  - [Preview] Speculative decoding
  - Add Python binding for `GptManager`
  - Add a Python class `ModelRunnerCpp` that wraps C++ `gptSession`
  - System prompt caching
  - Enable split-k for weight-only cutlass kernels
  - FP8 KV cache support for XQA kernel
  - New Python builder API and `trtllm-build` command(already applied to blip2 and OPT)
  - Support `StoppingCriteria` and `LogitsProcessor` in Python generate API (thanks to the contribution from @zhang-ge-hao)
  - fMHA support for chunked attention and paged kv cache

Dec. 27 2023

TensorRT-LLM Usage, Continued

Example: Llama and Variants in TensorRT-LLM and Triton

- CodeLlama: with customized RoPE rotary base.

```
# Inside container and mount to TRTLLM backend source
docker run --rm --gpus all --ipc=host -v $WORKSPACE:/wsp triton_trt_llm \
  bash -c  "cd /wsp/tensorrtllm_backend/tensorrt_llm/examples/llama && \
            python example/llama/build.py
            --model_dir /wsp/cache/codellama-13b-Instruct-hf \
            --dtype float16 \
            --use_inflight_batching \
            --use_gemm_plugin float16 \
            --enable_context_fmha \
            --rotary_base 1000000 \
            --max_input_len $MAX_INPUT_LEN \
            --max_output_len $MAX_OUTPUT_LEN \
            --max_batch_size $MAX_BATCH_SIZE \
            --gpus_per_node 1 \
            --output_dir /wsp/engine/codellama-13b-Instruct-hf/fp16/1-gpu"
```

Engine Build

```
...
# In TensorRT-LLM backend folder
# Copy a model repository template from `all_models/inflight_batcher_llm` to `all_models/codellama_repo_ifb`
python3 tools/fill_template.py \
        -i all_models/codellama_repo_ifb/ensemble/config.pbtxt \
        triton_max_batch_size:$MAX_BATCH_SIZE

python3 tools/fill_template.py \
        -i all_models/codellama_repo_ifb/tensorrt_llm/config.pbtxt \
        engine_dir:/engine/codellama-13b-Instruct-hf/fp16/1-gpu,decoupled_mode:True,triton_max_batch_size:$MAX_BATCH_SIZE,\
batching_strategy:inflight_fused_batching,batch_scheduler_policy:max_utilization,kv_cache_free_gpu_mem_fraction:0.5,max_queue_delay_microseconds:5
00

python3 tools/fill_template.py \
        -i all_models/codellama_repo_ifb/preprocessing/config.pbtxt \
        tokenizer_dir:/cache/codellama-13b-Instruct-hf,tokenizer_type:auto,triton_max_batch_size:$MAX_BATCH_SIZE,preprocessing_instance_count:1

python3 tools/fill_template.py \
        -i all_models/codellama_repo_ifb/postprocessing/config.pbtxt \
        tokenizer_dir:/cache/codellama-13b-Instruct-hf,tokenizer_type:auto,triton_max_batch_size:$MAX_BATCH_SIZE,postprocessing_instance_count:1
```

Setup Variables in Triton Model Repository

TensorRT-LLM Usage, Continued

Example: Llama and Variants in TensorRT-LLM and Triton

- Mistral: with similar build to Llama-2 but run with sliding window size

```
# Inside container and mount to TRTLLM backend source
docker run --rm --gpus all --ipc=host -v $WORKSPACE:/wsp triton_trt_llm \
  bash -c  "cd /wsp/tensorrtllm_backend/tensorrt_llm/examples/llama && \
            python examples/llama/build.py \
            --model_dir /wsp/cache/mistral-7b-0.1 \
            --dtype float16 \
            --use_inflight_batching \
            --use_gemm_plugin float16 \
            --enable_context_fmha \
            --max_input_len 32256 \
            --max_output_len 512 \
            --max_batch_size $MAX_BATCH_SIZE \
            --gpus_per_node 1 \
            --output_dir /wsp/engine/mistral-7b-0.1/1-gpu"
```

Engine Build

```
...
# In TensorRT-LLM backend folder
# Copy a model repository template from `all_models/inflight_batcher_llm` to `all_models/mistral_repo_ifb`
python3 tools/fill_template.py \
        -i all_models/mistral_repo_ifb/ensemble/config.pbtxt \
        triton_max_batch_size:$MAX_BATCH_SIZE

python3 tools/fill_template.py \
        -i all_models/mistral_repo_ifb/tensorrt_llm/config.pbtxt \
        engine_dir:/wsp/engine/mistral-7b-0.1/1-
gpu,decoupled_mode:True,triton_max_batch_size:$MAX_BATCH_SIZE,batching_strategy:inflight_fused_batching,\
batch_scheduler_policy:max_utilization,max_queue_delay_microseconds:500,max_attention_window_size:4096

python3 tools/fill_template.py \
        -i all_models/mistral_repo_ifb/preprocessing/config.pbtxt \
        tokenizer_dir:/cache/mistral-7b-0.1,tokenizer_type:auto,triton_max_batch_size:$MAX_BATCH_SIZE,preprocessing_instance_count:1

python3 tools/fill_template.py \
        -i all_models/mistral_repo_ifb/postprocessing/config.pbtxt \
        tokenizer_dir:/cache/mistral-7b-0.1,tokenizer_type:auto,triton_max_batch_size:$MAX_BATCH_SIZE,postprocessing_instance_count:1
```

Setup Variables in Triton Model Repository

TensorRT-LLM Usage, Continued

Example: Llama and Variants in TensorRT-LLM and Triton

- Mixtral-8x7B: with Mixture of Experts

```
# TRTLLM backend cloned to $WORKSPACE/tensorrtllm_backend
docker run --rm --gpus all --ipc=host -v $WORKSPACE:/wsp triton_trt_llm \
  bash -c  "cd /wsp/tensorrtllm_backend/tensorrt_llm/examples/llama && \
            pip install -r ../mixtral/requirements.txt && \
            python3 build.py \
            --model_dir /wsp/cache/Mixtral-8x7B-v0.1 \
            --dtype float16 \
            --use_inflight_batching \
            --use_gemm_plugin float16 \
            --enable_context_fmha \
            --max_input_len 128 \
            --max_output_len 128 \
            --max_batch_size 1 \
            --gpus_per_node 4 \
            --parallel_build \
            --world_size 4 \
            --tp_size 4 \
            --output_dir /wsp/engine/mixtral/fp16/4-gpu/"
```

Engine Build

```
...
# In TensorRT-LLM backend folder
# Copy a model repository template from `all_models/inflight_batcher_llm` to `all_models/mixtral_repo_ifb`
python3 tools/fill_template.py \
        -i all_models/mixtral_repo_ifb/ensemble/config.pbtxt \
        triton_max_batch_size:$MAX_BATCH_SIZE

python3 tools/fill_template.py \
        -i all_models/mixtral_repo_ifb/tensorrt_llm/config.pbtxt \
        engine_dir:/wsp/engine/mixtral/fp16/4-gpu/,decoupled_mode:True,triton_max_batch_size:1,batching_strategy:inflight_fused_batching,\
batch_scheduler_policy:max_utilization,max_queue_delay_microseconds:500

python3 tools/fill_template.py \
        -i all_models/mixtral_repo_ifb/preprocessing/config.pbtxt \
        tokenizer_dir:/cache/Mixtral-8x7B-v0.1,tokenizer_type:auto,triton_max_batch_size:$MAX_BATCH_SIZE,preprocessing_instance_count:1

python3 tools/fill_template.py \
        -i all_models/mixtral_repo_ifb/postprocessing/config.pbtxt \
        tokenizer_dir:/cache/Mixtral-8x7B-v0.1,tokenizer_type:auto,triton_max_batch_size:$MAX_BATCH_SIZE,postprocessing_instance_count:1
```

Setup Variables in Triton Model Repository

TensorRT-LLM Usage, Continued

Example: LoRA with Llama-2

- Pull LoRA weight from HuggingFace
- Build model engine with LoRA plugin
- Run with base model and LoRA weight
  - Tokenizers of some LoRA models may be different to the base model ones. Please specify the correct one if needed.

```
python

>>> from huggingface_hub import snapshot_download
>>> snapshot_download(repo_id="oliverwang15/FinGPT_v32_Llama2_Sentiment_Instructio
n_LoRA_FT", local_dir="/wsp/llama2-7b-hf-fin-sent/", local_dir_use_symlinks=False)
```

```
python build.py --model_dir /wsp/cache/Llama-2-7b-chat-hf/ \
                --dtype float16 \
                --remove_input_padding \
                --use_gpt_attention_plugin float16 \
                --enable_context_fmha \
                --use_gemm_plugin float16 \
                --output_dir "/wsp/engine/llama2-7b-hf-fin-sent/fp16/1-gpu" \
                --max_batch_size 4 \
                --max_input_len 2048 \
                --max_output_len 512 \
                --use_lora_plugin float16 \
                --hf_lora_dir "/wsp/cache/llama2-7b-hf-fin-sent/" \
                --use_fused_mlp \
                --gpus_per_node 1
```

```
python ../run.py --engine_dir "/wsp/engine/llama2-7b-hf-fin-sent/fp16/1-gpu" \
              --max_output_len 256 \
              --tokenizer_dir "/wsp/cache/Llama-2-7b-chat-hf/" \
              --input_text "Instruction: What is the sentiment of this news?
Please choose an answer from {negative/neutral/positive}\nInput: FINANCING OF
ASPOCOMP 'S GROWTH Aspocomp is aggressively pursuing its growth strategy by
increasingly focusing on technologically more demanding HDI printed circuit boards
PCBs.\nAnswer: " \
              --lora_dir "/wsp/cache/llama2-7b-hf-fin-sent" \
              --lora_task_uids 0 \
              --no_add_special_tokens
```

Upcoming Event

首次揭秘！
利用大模型辅助设计 NVIDIA 芯片

2024年1月20日 19:30-20:15 线上

特别为大模型的入门开发者量身定制!

- **ChipNeMo 介绍 —— 探索大模型在工业芯片设计中的应用**
- 演讲提纲：
  - ChipNeMo：探索 LLMs 在工业芯片设计中的应用
  - Domain Adaptation 技术详解
  - LLMs应用评估：聊天机器人、EDA脚本生成、缺陷分析
  - 实验结果：LLM 性能明显优于基础模型，并且领域适应和 RAG 是独立的，即使采用 RAG，也比 GPT-4 高出 20%
  - 未来展望
- 听众收益：
  - 如何将大模型应用在工业芯片设计中？
  - Domain Adaptation技术的详解与应用
  - 如何评估大模型在芯片设计场景中的效果？

NVIDIA
设计自动化研究总监
IEEE Fellow
任昊星 (Mark)